

In-Memory DataBase (IMDB)

In-Memory DataBase

Как это обычно бывает, у термина In-Memory DataBase нет устойчивого русского перевода, это что-то вроде «базы данных в оперативной памяти».

Базы данных in-memory хранятся в оперативной памяти сервера. Очевидно, что это обеспечивает большие преимущества в скорости, поскольку операции с данными в памяти выполняются меньшим числом инструкций процессора

In-Memory DataBase

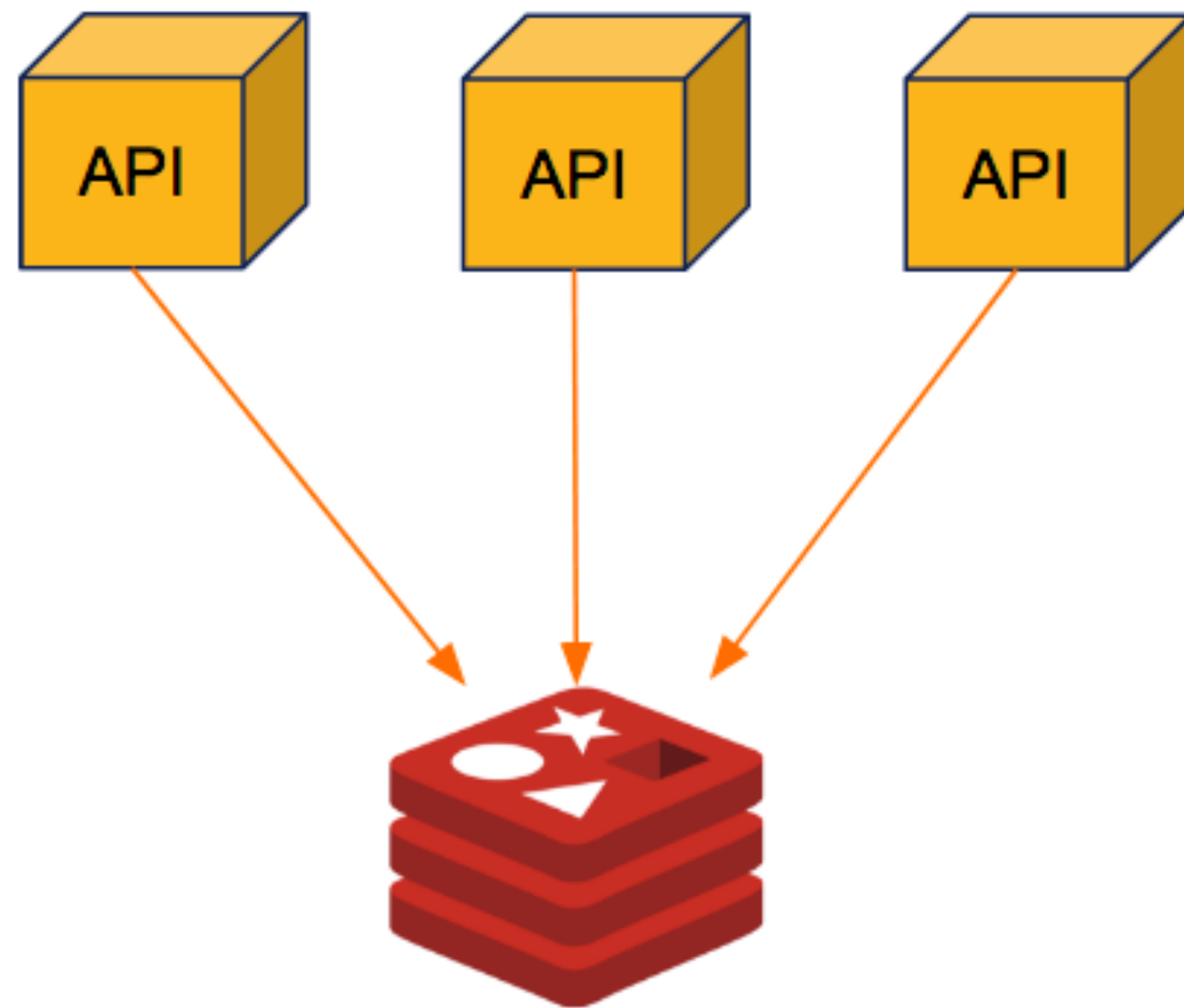
В терминах ACID базы данных in-memory конечно жертвуют надежностью, ведь при обесточивании или перезагрузке в них пропадают все данные.

Существуют ряд методов, которые позволяют снизить этот риск:

- Снэпшоты
- Логи транзакций
- Использование NVRAM или NVDIMM

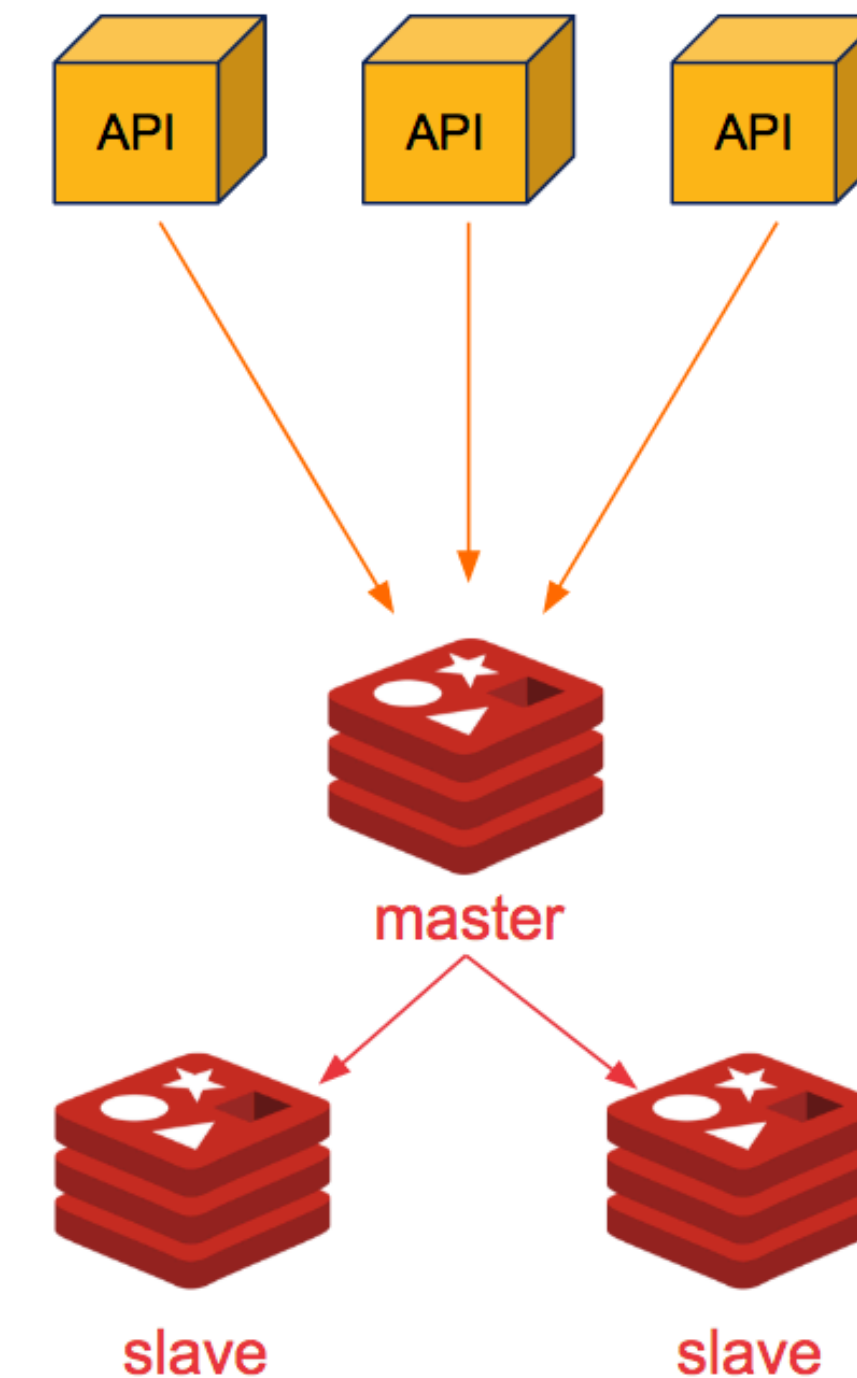
Топология Redis

Один Redis-инстанс



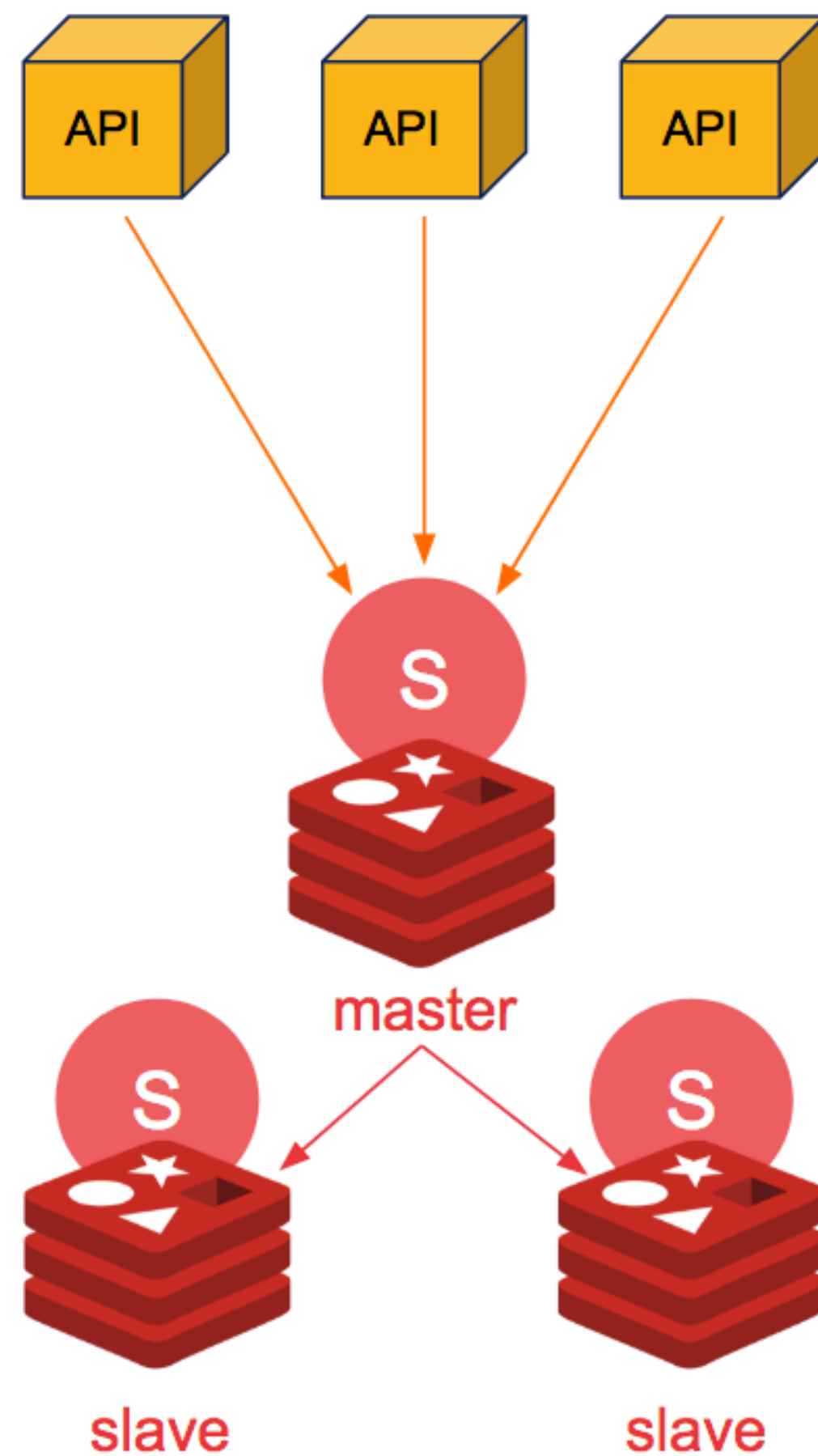
Имеет право на существование, но исключительно для сред разработки и тестирования, поскольку не является отказоустойчивой.

Master-slave репликация



В такой топологии между мастером и репликой происходит постоянная асинхронная репликация

Топология Redis Sentinel

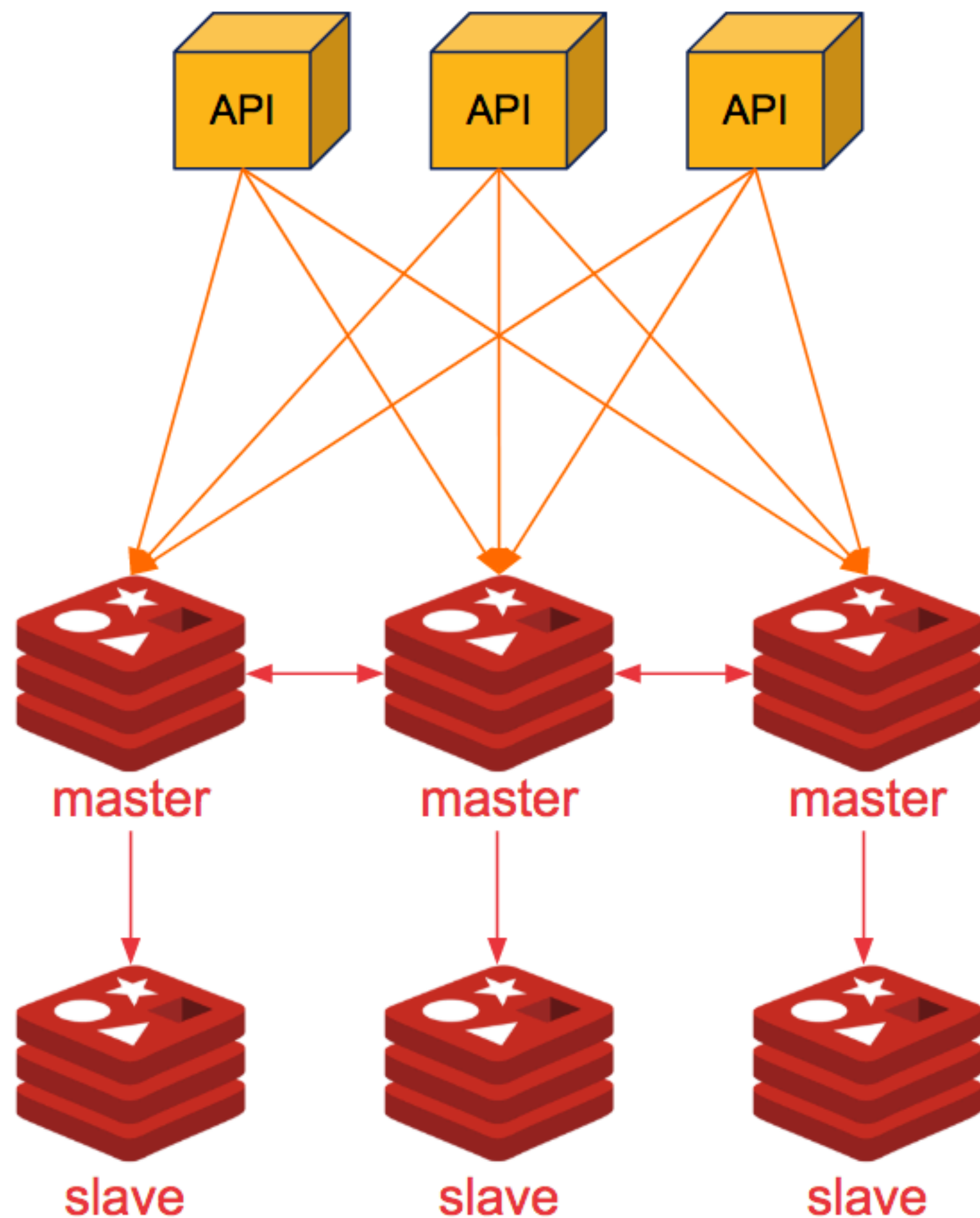


Эта топология развёртывания широко применялась на ранних версиях Redis, до поддержки полноценного кластера.

Она состоит из отдельных специальных узлов, которые мониторят работу основных узлов Redis, отслеживают сбои Master и запускают процесс восстановления.

Работает поверх топологии Master-Replica.

Топология Redis Cluster



Для репликации используется протокол Gossip: каждый узел передает информацию известным ему «соседям». Клиенты могут работать как с мастером, так и с репликой, но с реплики идет только чтение.

Данные в кластере шардированы. Кластер использует хеш-слоты. Всего кластер имеет 16384 слота. Каждый узел Redis отвечает за конкретное подмножество хеш-слотов.

Например:

- Узел А содержит хеш-слоты от 0 до 5500
- Узел В содержит хеш-слоты от 5501 до 11001
- Узел С содержит хеш-слоты от 11001 до 16383

Redis. key-value архитектура

Redis — быстрое хранилище в памяти с открытым исходным кодом для структур данных «ключ-значение». Redis поставляется с набором разнообразных структур данных в памяти, что упрощает создание различных специальных приложений.

Несколько фактов о ключах:

- Плохая идея - хранить слишком длинные ключи.
- Хорошая идея — придерживаться схемы при построении ключей: «object-type:id:field».

Типы данных

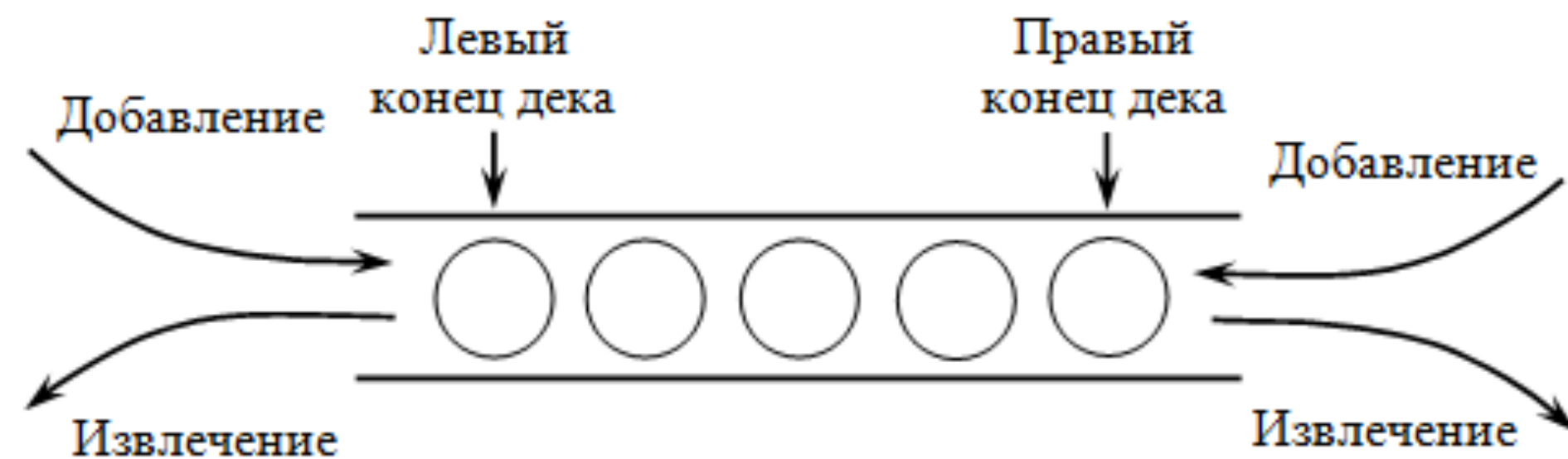
- **Строки** (strings) - базовый тип, ограничен размером 512 Мб.
- **Списки** (lists) - максимальное количество элементов - $2^{32} - 1$
- **Множества** (sets) - максимальное количество элементов - $2^{32} - 1$
- **Хэш-таблицы** (hashes) - максимальное количество элементов - $2^{32} - 1$
- **Упорядоченные множества** (sorted sets) - элементы упорядочены по параметру «score»

Базовые команды для Redis

- **SET** - Команда используется для установки ключа и его значения, с дополнительными необязательными параметрами для указания срока действия записи значения ключа
- **TTL** - Когда ключ установлен с истечением срока действия
- **GET** - Команда используется для получения значения, связанного с ключом
- **EXISTS** - Команда проверяет, существует ли что то с данным ключом. Она возвращает 1 если объект существует или 0 если нет
- **FLUSHALL** - Команда полностью удаляет все данные в текущем сеансе
- **DEL** - Команда удаляет ключ и соответствующее значение
- **KEYS** - Возвращает все ключи из базы по указанному шаблону
- **INCR / DECR** - Инкремент / декримент

Список

Список это последовательность значений упорядоченных по порядку их создания. Список в Redis - аналог двух-стороннего стека и применяется обычно для создания очередей.



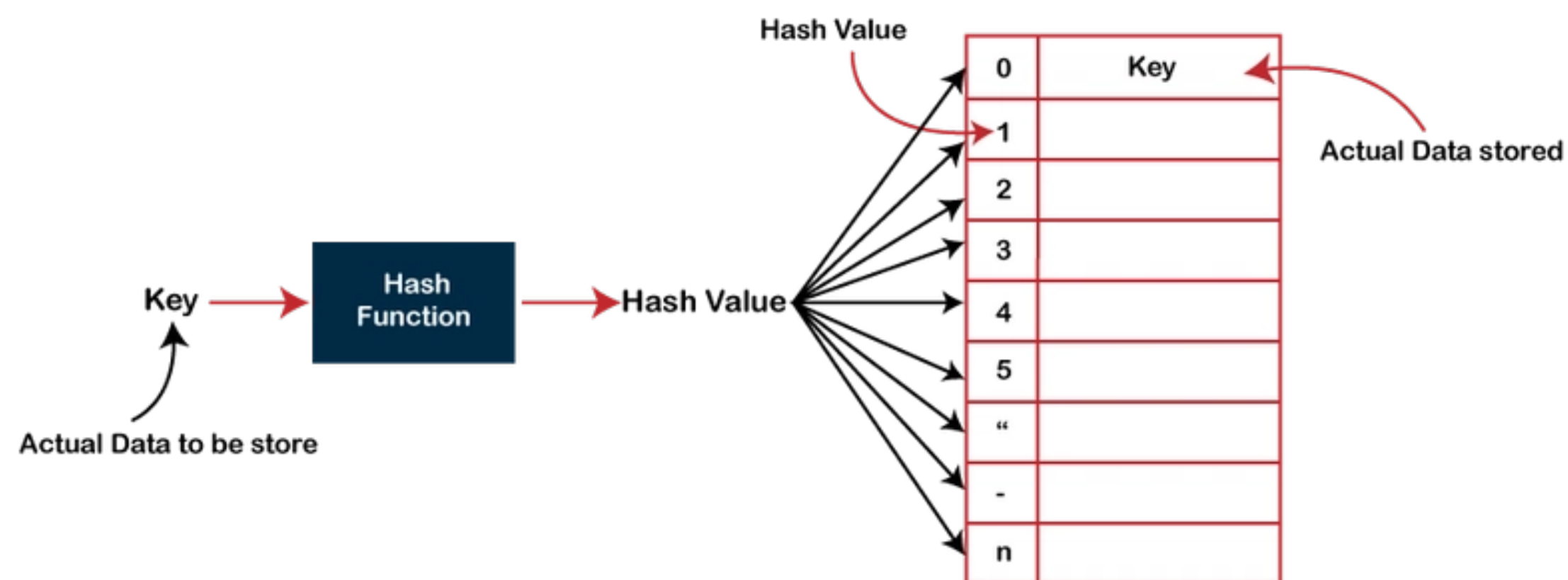
Базовые команды:

- LPUSH — Команда добавления элемента в список
- LRANGE — возвращает срез списка с левой стороны
- LPOP — вернуть одно значение с левой стороны и удалить его из списка

Команды RPUSH, RRANGE, RPOP аналогичны, но только для правой стороны.

Хэш-таблица

Redis позволяет в качестве значения так же использовать ключ: значение. Что по сути будет почти аналогией объектов из JavaScript или словари в Python.



Базовые команды:

- Для записи объекта используется команда HSET в следующем формате HSET имя_ключа имя_атрибута значение
- Для чтения объекта используется команда HGET в формате HGET имя_ключа имя_атрибута
- Команда HGETALL используется для получения всей хэш-таблицы

Механизм подписок PUS-SUB

Одно из основных преимуществ Redis от других key-value хранилищ заключается в том, что в Redis есть механизм подписок.

Redis можно использовать как сервер сообщений.

Одни клиенты подписываются на определенные каналы используя команду SUBSCRIBE имя_канала:

```
SUBSCRIBE channel
```

Другие клиенты могут отправлять сообщения в этот канал используя команду PUBLISH имя_канала значения:

```
PUBLISH channel "hello world"
```

Python и Redis

Redis очень широко применяется в современной разработке ПО.

Библиотеки поддержки есть для любого языка программирования.

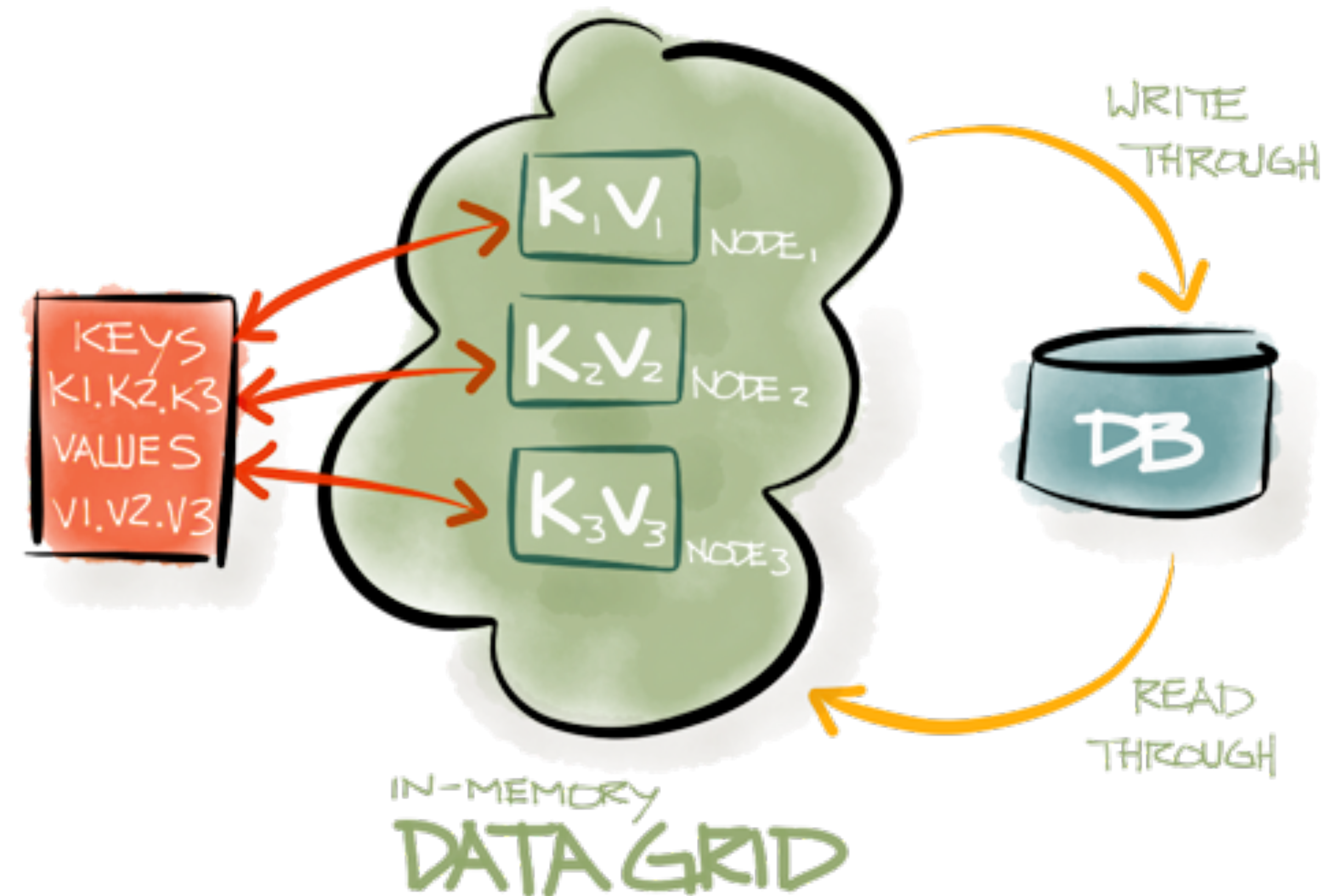
Рассмотрим python библиотеку Redis.

```
from time import sleep
import redis
```

```
with redis.Redis(host='127.0.0.1') as client:
    client.set('Language', 'Python', ex=5)
    print(client.get('Language'))
    sleep(3)
    print(client.ttl('Language'))
    sleep(3)
    print(client.ttl('Language'))
```


Преимущества In-Memory DataBase

- Высокая производительность
- Структуры данных в памяти
- Универсальность и простота использования
- Репликация и сохранность
- Поддержка удобного языка разработки



Примеры использования In-Memory DataBase

- Кэширование
- Управление сессиями
- Таблицы лидеров в режиме реального времени
- Ограничение интенсивности
- Очереди
- Чат и обмен сообщениями

